

STUDY WITH ME

Azure Data Fundamentals

Microsoft Certified

By Nicolý Oliveira

CONTEÚDOS ABORDADOS

1

Descrever conceitos relacionais



Identificar recursos de dados relacionais



Descreva a normalização e por que ela é usada



Identificar instruções SQL (linguagem de consulta estruturada) comuns



Identificar objetos comuns de banco de dados

2

Descrever os serviços de dados relacionais do Azure



Descrever a família de produtos SQL do Azure, incluindo o SQL do Azure Banco de dados, Instância Gerenciada de SQL do Azure e SQL Server no Azure Virtual Máquinas



Identificar serviços de banco de dados do Azure para sistemas de banco de dados de software livre

01.

**Descrever conceitos
relacionais**

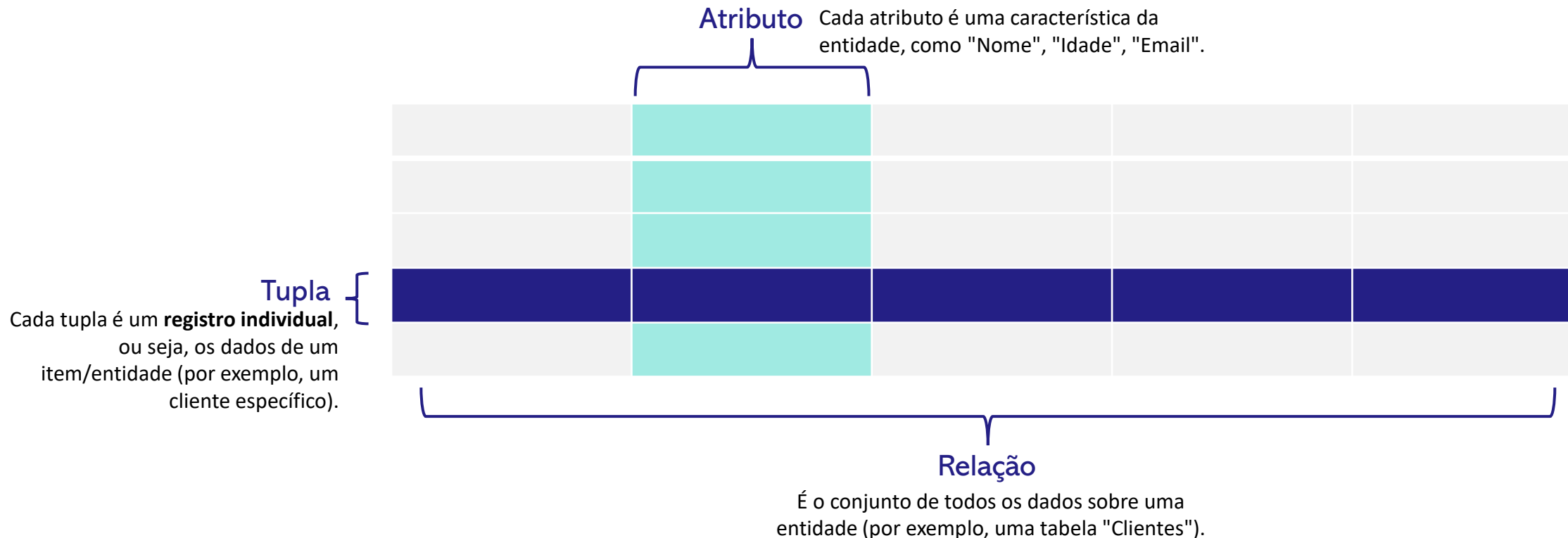
Descrever conceitos relacionais

Banco de dados relacional

Um banco de dados relacional é um tipo de banco de dados que **armazena e fornece acesso a pontos de dados relacionados entre si**. Baseado no modelo relacional, uma maneira intuitiva e direta de representar dados em tabelas.

Cada linha na tabela é um registro com uma ID exclusiva chamada Chave.

As colunas e tabelas **contêm atributos** dos dados e cada registro geralmente **tem um valor para cada atributo**, facilitando o estabelecimento das relações entre os pontos de dados



Descrever conceitos relacionais

Um exemplo de banco de dados relacional

Exemplo simples de duas tabelas que uma empresa pode usar para processar pedidos de seus produtos. A primeira tabela é de informações do cliente (Nome, endereço, envio, faturamento) Cada atributo de informação esta em sua própria coluna, e o banco de dados atribui uma ID única (Chave) a cada linha.

Essas duas tabelas tem apenas uma coisa em comum, **a coluna ID**. Mas por causa dessa coluna comum, o banco de dados relacional pode criar uma relação entre as duas tabelas.

Cliente				
#	Nome	Endereço	Envio	Faturamento
1	Maria Santos	Rua A, 123	SP	SP
2	João Souza	Rua B, 456	RJ	RJ
3	Ana Oliveira	Rua C, 789	MG	MG

Pedido			
ID	Data	Produto	Quantidade
1	2024 -03-01	Mouse	2
2	2024-03-05	Teclado	1
3	2024-03-07	Monitor	1

Descrever conceitos relacionais

Como os bancos de dados relacionais são estruturados

O modelo relacional significa que as estruturas de dados lógicas: tabelas de dados, exibições e índices - **são separadas das estruturas de armazenamento físico**. Essa separação significa que os administradores de banco de dados podem gerenciar o armazenamento de dados físicos sem afetar o acesso a esses dados como uma estrutura lógica.

As operações lógicas permitem que um aplicativo especifique o conteúdo necessário e as operações físicas determinam como esses dados devem ser acessados e, em seguida executa a tarefa

Tipo de Operação	O que faz	Exemplo
Lógica	Diz o que você quer dos dados	SELECT nome FROM clientes WHERE cidade = 'SP'
Física	Define como os dados serão lidos	Buscar no índice, fazer varredura, acessar páginas em disco, etc.

Descrever conceitos relacionais

Modelagem

Para criar um banco de dados relacional, é importante entender as diferenças entre modelagem lógica e modelagem física. A modelagem lógica descreve as necessidades de negócios e não envolve projetar um banco de dados. Já a modelagem física é necessária para o projeto real do banco de dados e inclui índices e constraints.

Características	Conceitual	Lógico	Físico
Nome da entidade	✓	✓	
Relacionamentos de Entidade	✓	✓	
Atributos	✓	✓	
Chave Primária	✓	✓	✓
Chave estrangeira		✓	✓
Nome das tabelas			✓
Nome das colunas			✓
Tipo das colunas			✓

Descrever conceitos relacionais

Normalização

A Normalização é focada na prevenção de problemas com repetição e atualização de dados, assim como o cuidado com a integridade. Esse processo busca anomalias, isto é, repetições e redundâncias entre os dados. Ao identificar, anomalias, o conjunto de regras da normalização tentará eliminá-las e redefinir relações afetadas.

Formas Normais

Seguindo esse conceito de padronização, temos regras estruturadas e agrupadas em 4 níveis. Esses grupos são denominados formas normais. Cada forma normal segue requisitos da forma anterior, ou seja, se mantém uma herança de requisitos, com exceção da primeira forma que não possui uma antecessora.

Descrever conceitos relacionais

Normalização

1 Forma

Nesta primeira forma **tratamos as repetições**, e também nos certificamos que os **atributos estão sendo armazenados de forma única**, isto é, não há nenhum outro atributo com os valores da mesma linha na tabela. A princípio, **identificamos a chave primária**, neste caso, é o ****atributo de código****. Considere que entre os atributos estão os valores associados a chave, então precisamos fragmentar estes valores, desta forma é necessário criar uma nova tabela.

Código	Nome	Localização	Telefone
1	José	Curitiba	(44) 91234-5678
		Paraná	(44) 94834-8348
2	Arturo	Recife	(81) 91234-8765
		Pernambuco	(81) 91328-7811



Criamos os atributos cidade e estado, porque estas informações **estavam em um único atributo**, sendo que **é mais útil ter essas informações separadas**, para filtrar os dados por exemplo.

Código	Nome	Cidade	Estado
1	José	Curitiba	Paraná
2	Arturo	Recife	Pernambuco



Código	Telefone
1	(44) 91234-5678
2	(44) 94834-8348
3	(81) 91234-8765
4	(81) 91328-7811

Esta nova tabela foi criada para poder **relacionar telefones com o atributo código**, que na tabela principal é a chave primária, sendo definida como chave estrangeira. Deixamos todos os dados definidos individualmente, ainda assim relacionados.

Descrever conceitos relacionais

Normalização

2 Forma

A segunda forma trabalha focada nas possíveis **redundâncias nas tabelas**, em especial, **define se os atributos da tabela dependem inteiramente da chave primária**. Os atributos que não dependem ou dependem parcialmente da chave são associados a uma outra tabela, agora com uma relação clara com a chave primária da tabela original. Em outras palavras, a chave primária é convertida em chave estrangeira (ou externa) na nova tabela..

Código	Nome	Código Voo	Origem	Destino
1	José	101	Santiago	São Paulo
2	Arturo	102	Bogotá	Buenos Aires

Considere que os campos de origem e destino não têm relação direta com o campo de código, mas têm uma relação direta com o código de voo, já que são informações relacionadas a uma viagem aérea, por exemplo. Assim, podemos mover essas informações a uma nova tabela sem que os dados percam as relações originais

Código Voo	Origem	Destino
101	Santiago	São Paulo
102	Bogotá	Buenos Aires

Código	Nome	Código Voo
1	José	101
2	Arturo	102

Por isso, a tabela original elimina os dados que não são necessários nela, porém eles seguem relacionados em uma tabela secundária.

Descrever conceitos relacionais

Normalização

3 Forma

Na terceira forma normal trabalhamos precisamente com a organização dos **atributos que dependem uns dos outros, porém que não são atributos chaves** (primárias ou estrangeiras). Caso necessário, é criada uma tabela secundária para reestruturar a relação de dependência entre os atributos. Essas tabelas devem ter a chave primária ou estrangeira.

Placa	Modelo	Ano	Código Fabricação	Nome Fabrica
123X	Modelo1	2016	1	A
156Y	Modelo2	2012	2	B

Tenha em conta que existe uma dependência entre o atributo '**nome de fábrica**' e '**ano**' com o '**código de fábrica**', entretanto, estes atributos não dependem da chave principal da tabela que é 'placa'.

Código Fabricação	Nome Fabrica	Ano
1	A	2016
2	B	2012

Placa	Modelo	Código Fabricação
123X	Modelo 1	1
456Y	Modelo 2	2

Neste caso, criamos uma nova tabela para relacionar o nome da fábrica com seu código, e também removemos as relações de dependência entre atributos que não são chaves da tabela original.

Descrever conceitos relacionais

Normalização

4 Forma

A quarta e última forma normal é focada em eliminar dependências multivariadas entre os atributos chave, isto é, se há mais atributos (além das chaves primárias ou estrangeiras) que se repetem na tabela. Se isso ocorrer, geramos novas tabelas para eliminar tal redundância e manter as relações entre os atributos.

Música	Artista	Álbum
Musica 1	Artista A	Álbum 1
Musica 1	Artista B	Álbum 2
Musica 2	Artista B	Álbum 1

Música	Álbum
Musica 1	Álbum 1
Musica 1	Álbum 2
Musica 2	Álbum 2

Música	Artista
Musica 1	Artista A
Musica 1	Artista B
Musica 2	Artista B

Leve em conta que o campo 'música' está relacionado com 'artista' e 'álbum', porém, o artista e o álbum podem não estar relacionados, porque sabemos que a mesma canção pode estar em vários álbuns diferentes e também pode ser cantada por diferentes artistas. **Por isso, o ideal é que se produza a divisão desta tabela e assim eliminar repetições entre os dados.**

E agora o campo de música é relacionado ao campo de artista..

Schemas

Star , Snowflake e Galaxy(Constellation).

Os Schemas Star, Snowflake e Galaxy são técnicas de modelagem de dados utilizada principalmente em banco de dados relacionais, especialmente em Data warehouse e sistemas de Business Intelligence. Essas estruturas organizam os dados em tabelas fatos e dimensões, permitindo consultas analíticas eficientes e facilidade na geração de relatórios. Enquanto o Star Schema prioriza simplicidade e desempenho, o Snowflake Schema normaliza as dimensões para reduzir redundância, e o Galaxy Schema conecta múltiplas tabelas fato compartilhando dimensões comuns, suportando análises mais complexas .



Em bancos não relacionais, como MongoDB ou Cassandra, esses schemas não são aplicáveis, pois a modelagem foca em documentos, chave-valor ou colunas, visando flexibilidade e escalabilidade ao invés de join entre fatos e dimensões.

Fato
ID_Venda
ID_Data
ID_Produto
ID_Cliente
Quantidade_Vendida
Valor_Total

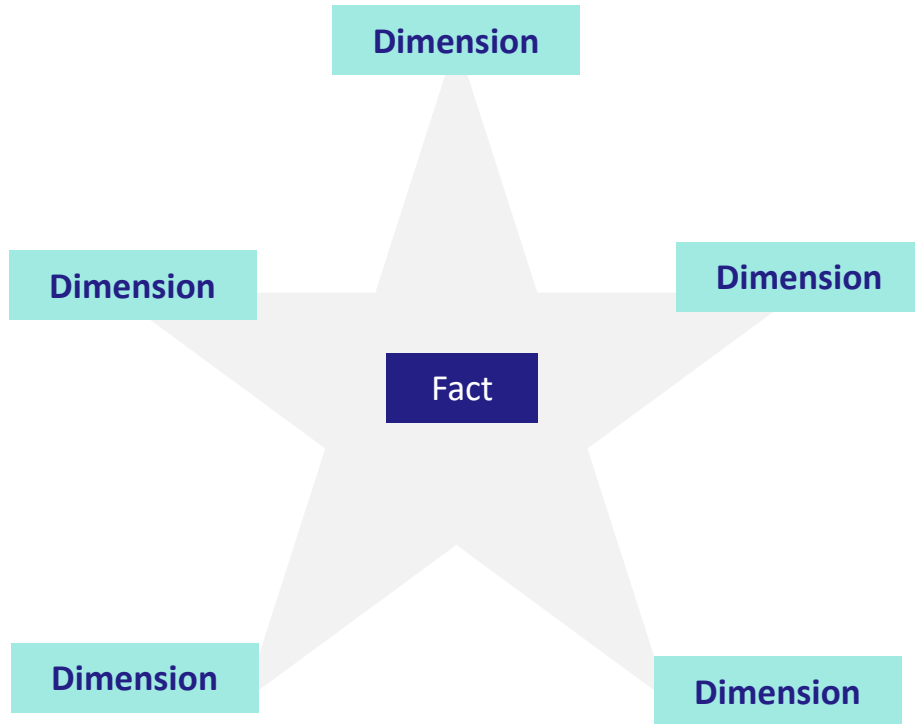
As duas últimas colunas (**quantidade e valor**) são os fatos (**métricas**)

É onde **ficam os dados quantitativos** (os números que podem ser medidos, somados, analisados).
Normalmente armazena **métricas como vendas, quantidade, valor, lucro, tempo gasto etc.**
Ela depende das tabelas de dimensão para dar contexto a esses números.

São tabelas de contexto, **usadas para descrever os fatos.**
Contêm informações qualitativas, como nomes, categorias, locais, atributos de tempo, etc.

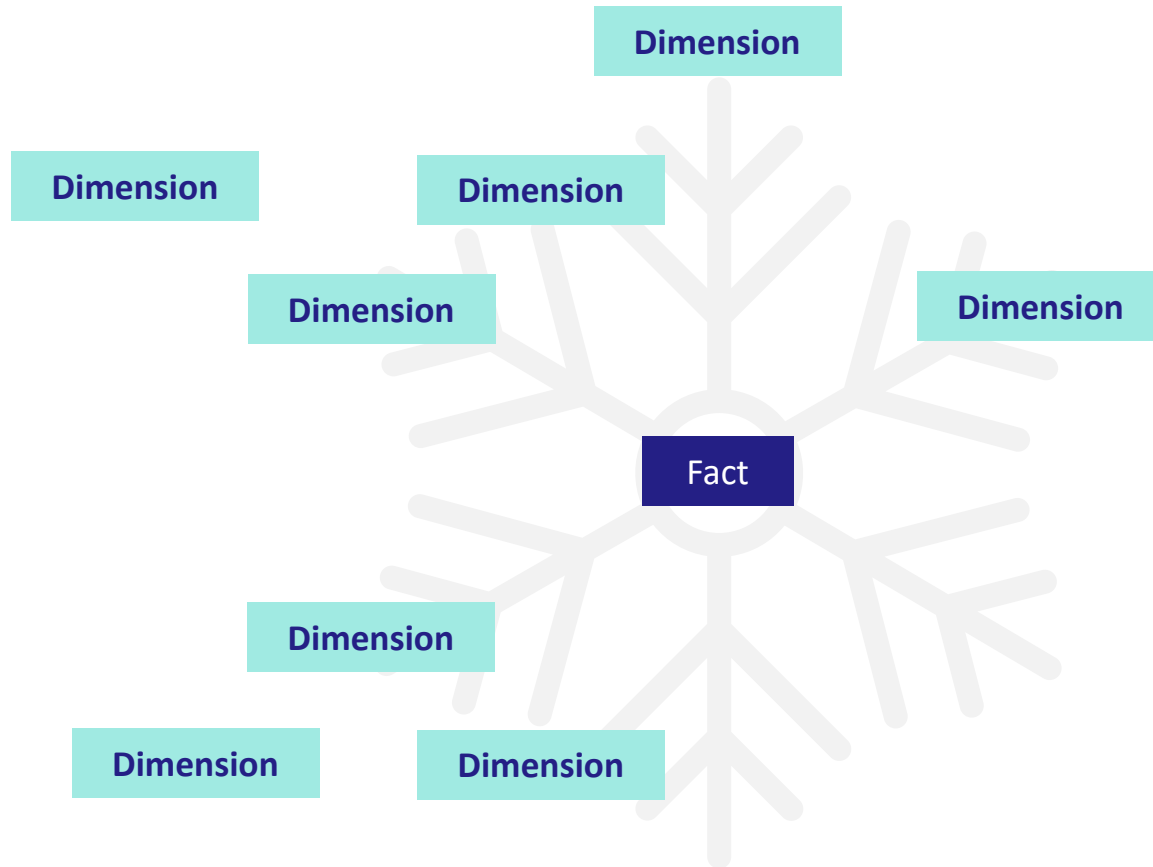
Dimensão
Dim_Produto > ID_Produto, Nome, Categoria, Marca
Dim_Cliente > ID_Cliente, Nome, Idade, Cidade
Dim_Data > ID_Data, Dia, Mês, Ano, Trimestre.

Schemas Star



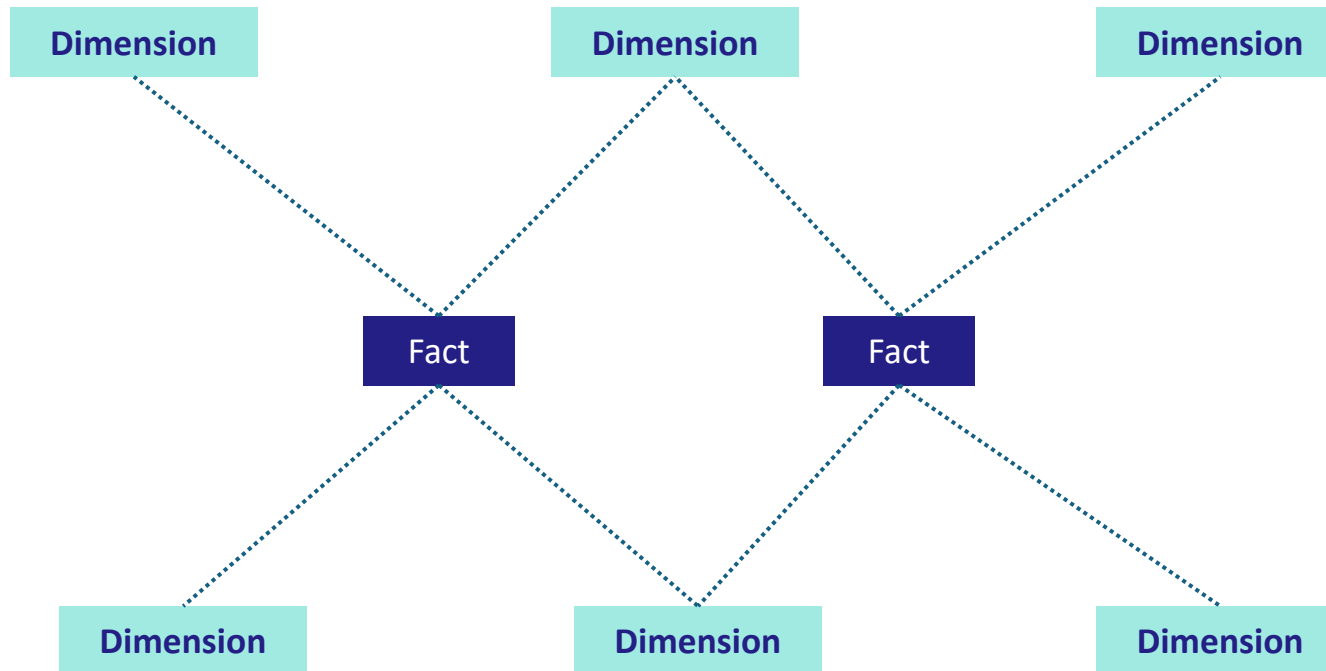
- Estrutura simples: **uma tabela fato** no centro ligada a várias **tabelas dimensão**.
- Fácil de entender e consultar.
- Bom desempenho para análises.
- Exemplo: Vendas (Fato) conectada a Cliente, Produto, Tempo, Local.

Schemas Snowflake



- É uma versão normalizada do Star.
- As dimensões são divididas em subdimensões, reduzindo redundância.
- Mais complexo, mas otimiza armazenamento.
- Exemplo: Dimensão Produto dividida em Categoria → Subcategoria → Produto.

Schemas Galaxy



- Também chamado de **Fact Constellation**.
- Contém **múltiplas tabelas fato** que compartilham dimensões.
- Suporta análises mais complexas, para grandes organizações.
- Exemplo: Fatos de Vendas e Encomendas usando dimensões comuns como Cliente, Tempo, Produto.

Identificar objetos comuns de banco de dados

Exibição

Uma exibição é uma tabela virtual com base no conjunto de resultados de uma **consulta SELECT**. Você pode considerar uma exibição como uma janela em linhas especificadas de uma ou mais tabelas subjacentes. Por exemplo, você pode criar uma exibição nas tabelas Order e Customer que recupera dados de pedidos e clientes para fornecer um único objeto que **facilita a determinação de endereços de entrega de pedidos**:

```
CREATE VIEW Deliveries
AS
SELECT o.OrderNo, o.OrderDate,
       c.FirstName, c.LastName, c.Address, c.City
FROM Order AS o JOIN Customer AS c
ON o.Customer = c.ID;
```

É possível consultar a exibição e filtrar os dados de maneira muito semelhante à de uma tabela. A consulta a seguir localiza detalhes de pedidos de clientes que vivem em Seattle:

```
SELECT OrderNo, OrderDate, LastName, Address
FROM Deliveries
WHERE City = 'Seattle';
```

Identificar objetos comuns de banco de dados

Procedimento Armazenado

Um procedimento armazenado define instruções SQL que podem ser executadas sob comando. Os procedimentos armazenados são usados para encapsular lógica programática de ações em um banco de dados que os aplicativos precisam executar ao trabalhar com os dados.

Você pode definir um procedimento armazenado com parâmetros para criar uma solução flexível para ações comuns que talvez precisem ser aplicadas aos dados com base em uma chave ou em critérios específicos. Por exemplo, o procedimento armazenado a seguir pode ser definido para alterar o nome de um produto com base na ID do produto especificada.

```
SQL ▾ Copiar Legenda ***  
  
CREATE PROCEDURE RenameProduct  
    @ProductID INT,  
    @NewName VARCHAR(20)  
AS  
UPDATE Product  
SET Name = @NewName  
WHERE ID = @ProductID;
```

Quando um produto precisa ser renomeado, você pode executar o procedimento armazenado, passando a ID do produto e o novo nome a ser atribuído:

```
SQL ▾ Copiar Legenda ***  
  
EXEC RenameProduct 201, 'Spanner';
```

Identificar objetos comuns de banco de dados

Índice

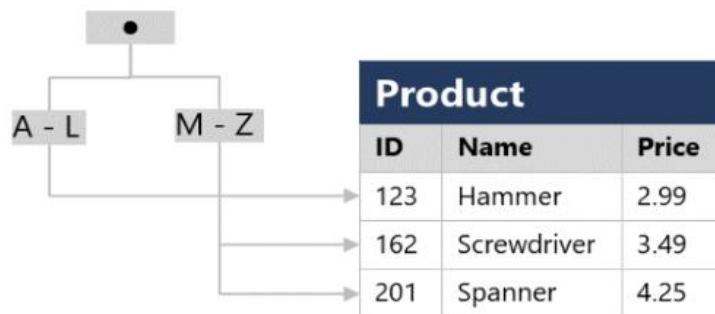
Um índice **ajuda a pesquisar dados em uma tabela**. Imagine um índice em uma tabela como um índice no final de um livro. Um índice de livro contém um conjunto classificado de referências, **com as páginas nas quais cada referência ocorre**. Quando você deseja encontrar uma referência a um item no livro, procura por ela no índice. Você pode usar os números de página no índice para ir diretamente para as páginas corretas no livro. Sem um índice, talvez seja necessário ler todo o livro para localizar as referências que você está procurando.

Quando você cria um índice em um banco de dados, **especifica uma coluna da tabela e o índice contém uma cópia desses dados em uma ordem classificada**, com ponteiros para as linhas correspondentes na tabela. Quando o usuário executa uma consulta que especifica essa coluna na cláusula ****WHERE****, o sistema de gerenciamento de banco de dados pode usar esse índice para buscar os dados mais rapidamente do que se precisasse examinar toda a tabela, linha por linha.

Por exemplo, você pode usar o código a seguir para criar um índice na coluna ****Name**** da tabela ****Product****:

```
SQL SQL Copiar Legenda ...  
  
CREATE INDEX idx_ProductName  
ON Product(Name);
```

O índice cria uma estrutura baseada em árvore que o otimizador de consulta do sistema do banco de dados pode usar para localizar rapidamente linhas na tabela Product com base em um Name especificado



Identificar objetos comuns de banco de dados

Trigger

Uma trigger executa automaticamente um conjunto de instruções no banco de dados em resposta a certos eventos em uma tabela, como inserção, atualização ou exclusão de registros. Imagine uma trigger como um alarme inteligente que dispara assim que algo acontece na tabela.

Você define o que deve acontecer quando o evento ocorre, e o banco de dados executa automaticamente essas ações, sem que você precise chamá-las manualmente. Quando você cria uma trigger, especifica a tabela e o evento que irá acioná-la.

A trigger pode então executar ações como atualizar outra tabela, validar dados, ou registrar alterações em um log. Isso ajuda a manter a integridade dos dados e automatizar regras de negócio sem depender de aplicativos externos.

Por exemplo, você pode usar o código a seguir para criar uma trigger que registra inserções na tabela Product em uma tabela de log:

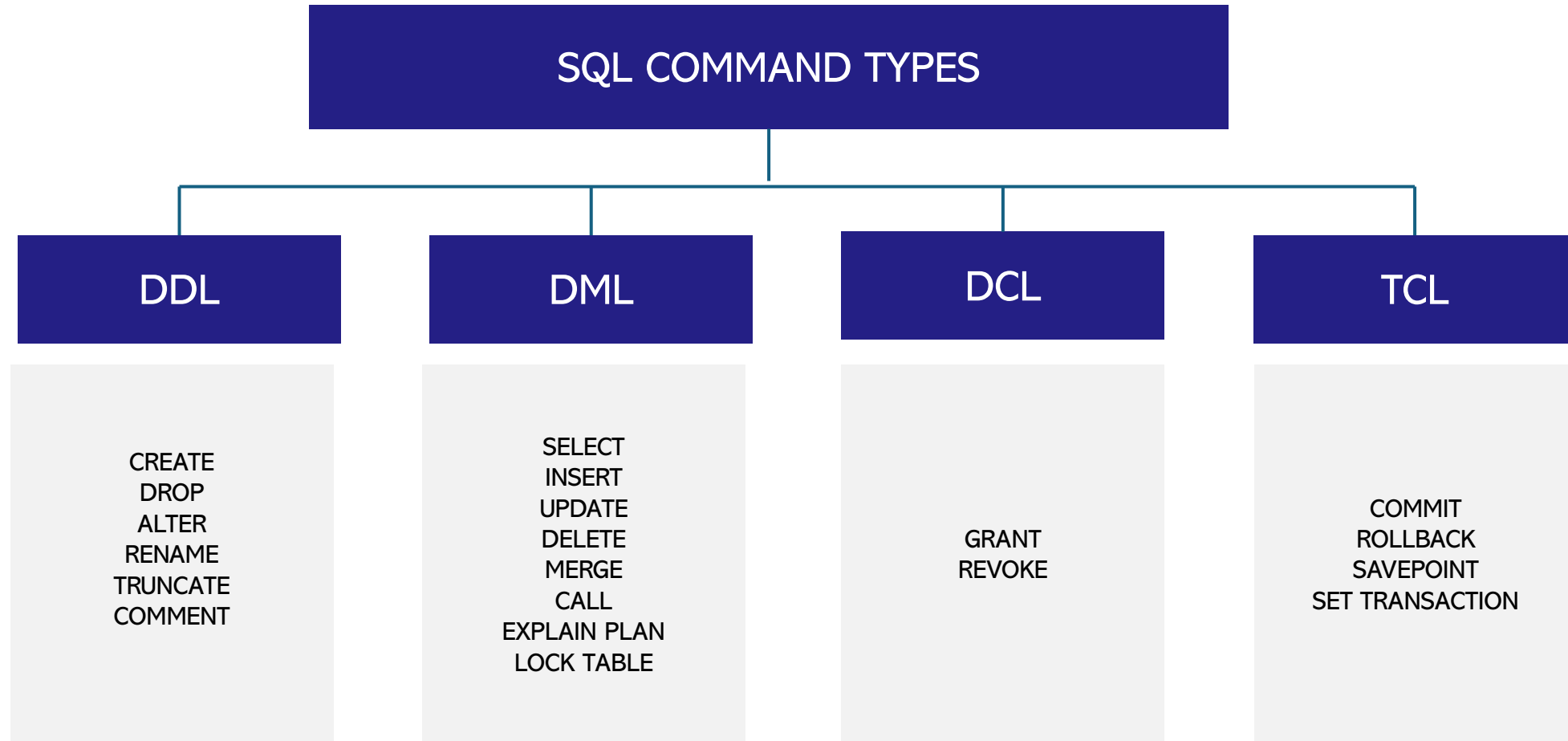
sql

 Copy code

```
CREATE TRIGGER trg_InsertProduct
ON Product
AFTER INSERT
AS
BEGIN
    INSERT INTO ProductLog (ProductID, Action, ActionDate)
    SELECT ProductID, 'INSERT', GETDATE()
    FROM inserted;
END;
```

Identificar instruções SQL (linguagem de consulta estruturada) comuns

DDL – Data Definition Language.



Identificar instruções SQL (linguagem de consulta estruturada) comuns

DDL – Data Definition Language.

A DDL é usada para **definir e modificar a estrutura** dos objetos no banco de dados, como tabelas, colunas, índices, e visões. Ela **não manipula dados**, apenas **estrutura**.

CREATE

Cria um novo objeto no banco de dados.

```
sql Copy Edit
CREATE TABLE Clientes (ID INT, Nome VARCHAR(100));
```

DROP

Exclui um objeto do banco permanentemente..

```
sql Copy Edit
DROP TABLE Clientes;
```

ALTER

Modifica a estrutura de um objeto existente.

```
sql Copy Edit
ALTER TABLE Clientes ADD Email VARCHAR(100);
```

RENAME

Renomeia um objeto no banco.

```
sql Copy Edit
RENAME TABLE Clientes TO Clientes_Ativos;
```

TRUNCATE

Remove todos os dados de uma tabela, mas mantém sua estrutura.

```
sql Copy Edit
TRUNCATE TABLE Clientes;
```

COMMENT

Adiciona uma descrição a objetos do banco, como tabelas ou colunas.

```
sql Copy Edit
COMMENT ON COLUMN Clientes.Nome IS 'Nome completo do cliente';
```

Identificar instruções SQL (linguagem de consulta estruturada) comuns

DML – Data Manipulation Language.

A DML é responsável por manipular os dados armazenados nas tabelas do banco. Com esses comandos, você pode **inserir, consultar, atualizar ou excluir** dados..

SELECT

Recupera dados de uma ou mais tabelas.

```
sql Copy Edit

SELECT * FROM Clientes WHERE Ativo = 1;
```

INSERT

Insere novos dados em uma tabela.

```
sql Copy Edit

INSERT INTO Clientes (Nome, Email) VALUES ('João', 'joao@email.com');
```

UPDATE

Altera dados existentes.

```
sql Copy Edit

UPDATE Clientes SET Email = 'novo@email.com' WHERE ID = 1;
```

DELETE

Remove registros da tabela.

```
sql Copy Edit

DELETE FROM Clientes WHERE ID = 1;
```

CALL

Executa uma procedure armazenada no banco.

```
sql Copy Edit

CALL AtualizaClientesInativos();
```

MERGE

Realiza upsert (insere ou atualiza) com base em uma condição.

```
sql Copy Edit

MERGE INTO Estoque E
USING NovosDados N ON (E.ID = N.ID)
WHEN MATCHED THEN UPDATE SET E.Qtd = N.Qtd
WHEN NOT MATCHED THEN INSERT (ID, Qtd) VALUES (N.ID, N.Qtd);
```

EXPLAN PLAN

Exibe o plano de execução de uma instrução SQL (usado para análise de desempenho).

```
sql Copy Edit

EXPLAIN PLAN FOR SELECT * FROM Clientes WHERE Nome LIKE 'A%';
```

LOCK TABLE

Bloqueia uma tabela para evitar alterações simultâneas por outros usuários.

```
sql Copy Edit

LOCK TABLE Clientes IN EXCLUSIVE MODE;
```

Identificar instruções SQL (linguagem de consulta estruturada) comuns

DCL – Data Control Language.

A DCL é usada para controlar o acesso aos dados e permissões dentro do banco. Ela define quem pode fazer o quê com os objetos do banco de dados.

GRANT

Concede permissões a usuários ou roles (funções).

sql

 Copy  Edit

```
GRANT SELECT, INSERT ON Clientes TO UsuarioX;
```

REVOKE

Remove permissões previamente concedidas.

sql

 Copy  Edit

```
REVOKE INSERT ON Clientes FROM UsuarioX;
```


Identificar instruções SQL (linguagem de consulta estruturada) comuns

TCL – Transaction Control Language

A DDL é usada para **definir e modificar a estrutura** dos objetos no banco de dados, como tabelas, colunas, índices, e visões. Ela **não manipula dados**, apenas **estrutura**.

COMMIT

Salva todas as alterações feitas na transação atual permanentemente no

sql

Copy Edit

```
COMMIT;
```

ROLLBACK

Desfaz todas as alterações feitas na transação atual.

sql

Copy Edit

```
ROLLBACK;
```

SAVEPOINT

Cria um ponto dentro da transação que pode ser usado para rollback parcial.

sql

Copy Edit

```
SAVEPOINT Ponto1;
```

ROLLBACK TO SAVEPOINT

Volta até um ponto salvo, sem desfazer toda a transação.

sql

Copy Edit

```
ROLLBACK TO Ponto1;
```

SET TRANSACTION

Define propriedades da transação atual (ex: nível de isolamento).

sql

Copy Edit

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

sql

Copy Edit

```
BEGIN;  
UPDATE Contas SET Saldo = Saldo - 100 WHERE ID = 1;  
SAVEPOINT Transf1;  
UPDATE Contas SET Saldo = Saldo + 100 WHERE ID = 2;  
  
-- Algo deu errado  
ROLLBACK TO Transf1;  
COMMIT;
```



Em um sistema real, se qualquer parte da transferência falhar, o ideal seria fazer um ROLLBACK total, para garantir que os dados fiquem consistentes (debita e credita, ou não faz nada). Mas o uso de SAVEPOINT é útil se você quiser desfazer apenas uma parte da transação.

02. **Descrever os serviços de dados relacionais do Azure**

Descrever conceitos relacionais

Um exemplo de banco de dados relacional

SQL SERVER em VMs do Azure	Instância Gerenciada de SQL	Banco de Dados SQL do Azure	SQL Do Azure Edge
Uma maquina virtual em execução no Azure com uma instalação do SQL Server. O Uso dessa VM torna uma solução de IaaS (Infraestrutura como serviço) que virtualiza a infraestrutura de hardware. Ótima opção para migração de instalações locais para a Nuvem.	Uma opção de PaaS (Plataforma como serviço) que fornece quase 100% de compatibilidade com instâncias de SQL server locais. Esse serviço inclui gerenciamento automatizado de atualizações de software, backups e outras tarefas de manutenção. Reduzindo a carga administrativa do suporte	Um serviço de banco de dados de PaaS totalmente gerenciado e altamente escalonável projetado para a nuvem. Esse sistema inclui principais recursos de nível banco de dados SQL Server local e é uma boa opção para aplicações em nuvem	Um mecanismo SQL que é otimizado para cenários de IoT (Internet das Coisas) que precisam trabalhar com transmissão de dados de séries temporais.



Use essa opção quando precisar migrar ou estender uma solução SQL Server local e **manter o controle total sobre todos os aspectos da configuração do servidor e do banco de dados**



Use essa opção para a maioria dos cenários de **migração na nuvem** especialmente quando você precisar de alterações mínimas nos aplicativos existentes.



Use essa opção para **novas soluções de nuvem** ou para migrar aplicativos que têm dependências mínimas no nível de instância.

Descrever conceitos relacionais

Um exemplo de banco de dados relacional

Além dos serviços de SQL do Azure, há serviços de dados do Azure disponíveis para outros sistemas de banco de dados relacionais populares, incluindo MySQL, MariaDB e PostgreSQL. Eles são sistemas de gerenciamento de banco de dados relacional personalizados para diferentes especializações.

My SQL	MariaDB	Postgre SQL
- Gerenciamento de banco de dados software livre - Principal banco de dados relacional para os aplicativos de pilha (Linux, Apache, MySQL e PHP) - Paga apenas pelo o que usar	- Gerenciamento de banco de dados mais recente - Recurso Notável é o suporte interno para dados temporais (guardar histórico)	- Banco de dados de objetos híbrido e relacional - Possibilidade de armazenar tipos de dados personalizados, com propriedades não relacionais próprias - Capacidade de armazenar dados geométricos, como linhas, círculos e polígonos. - Tem linguagem de consulta própria chamada pgsq